

LAB 1

DevOps Foundations & Continuous Integration

Task 2 - Prepare a CI/CD Workflow

Student: abhay

This report documents the working CI/CD workflow implemented with GitHub and Jenkins, including source checkout, pipeline stages, automated deployment, trigger configuration, and verification of the deployed application.

1. CI/CD Pipeline Architecture

Source Control: GitHub repository [abhayprajapati14042005/lab1-devops-version-control](#)

CI/CD Server: Jenkins job [abhay-Lab1-Basic-CICD](#)

Workflow: GitHub -> Jenkins checkout -> Build -> Test -> Deploy -> Post-build action -> Verified application

The GitHub repository is the source of the pipeline. Jenkins retrieves the repository and executes the declarative pipeline. The current workflow includes a real deployment step that copies the web application to a deployment directory and the deployed page is then verified through a local web server.

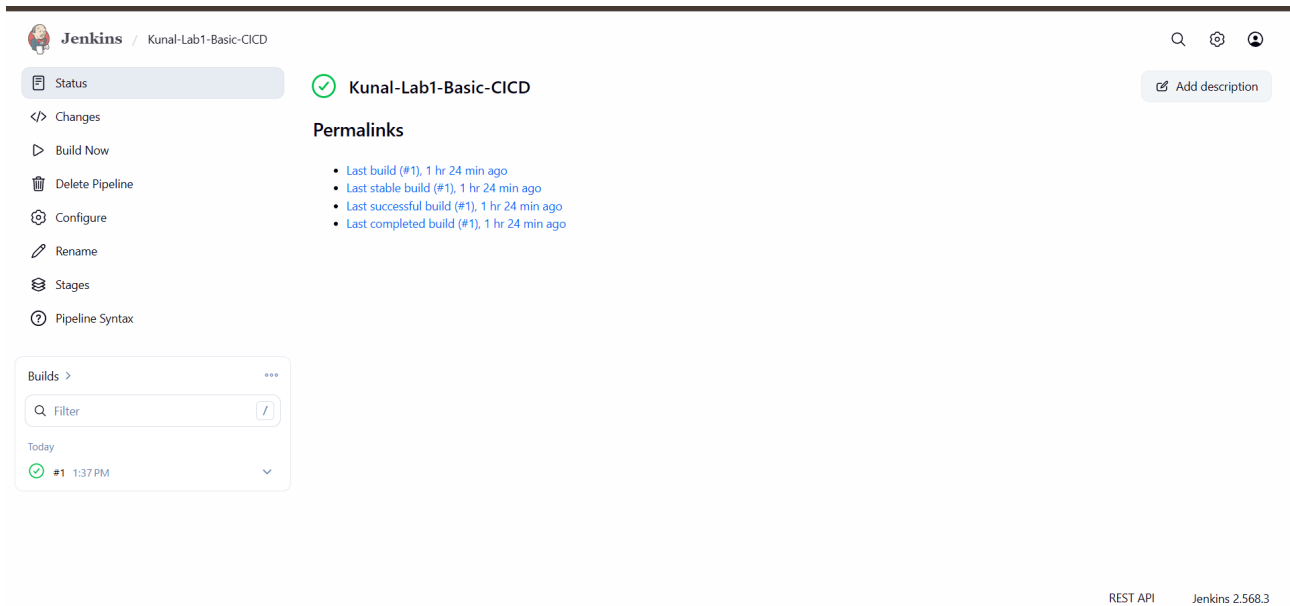


Figure 1. Jenkins job overview for abhay-Lab1-Basic-CICD.

2. Automated Build Trigger

Jenkins is configured to poll the Git repository using Poll SCM with the schedule H/5 * * * *. This provides an automated trigger that checks the repository for changes and starts the pipeline when a relevant change is detected.

Jenkins / Kunal-Lab1-Basic-CICD / Configure

Configure

- General
- Triggers**
- Pipeline
- Advanced

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☐ GitHub hook trigger for GITScm polling ?
- ☒ **Poll SCM ?**

Schedule ?

H/5 * * * *

Would last have run at Wednesday, September 9, 2026, 3:40:00 PM Coordinated Universal Time; would next run at Wednesday, September 9, 2026, 3:45:00 PM Coordinated Universal Time.

- ☐ Ignore post-commit hooks ?
- ☐ Trigger builds remotely (e.g., from scripts) ?

Pipeline

Save Apply

Figure 2. Jenkins Trigger configuration showing Poll SCM and the H/5 * * * * schedule.

3. Source Checkout and Pipeline Stages

Jenkins obtains the Jenkinsfile from the GitHub repository and checks out the main branch. The pipeline then executes the Build and Test stages before deployment.

Console

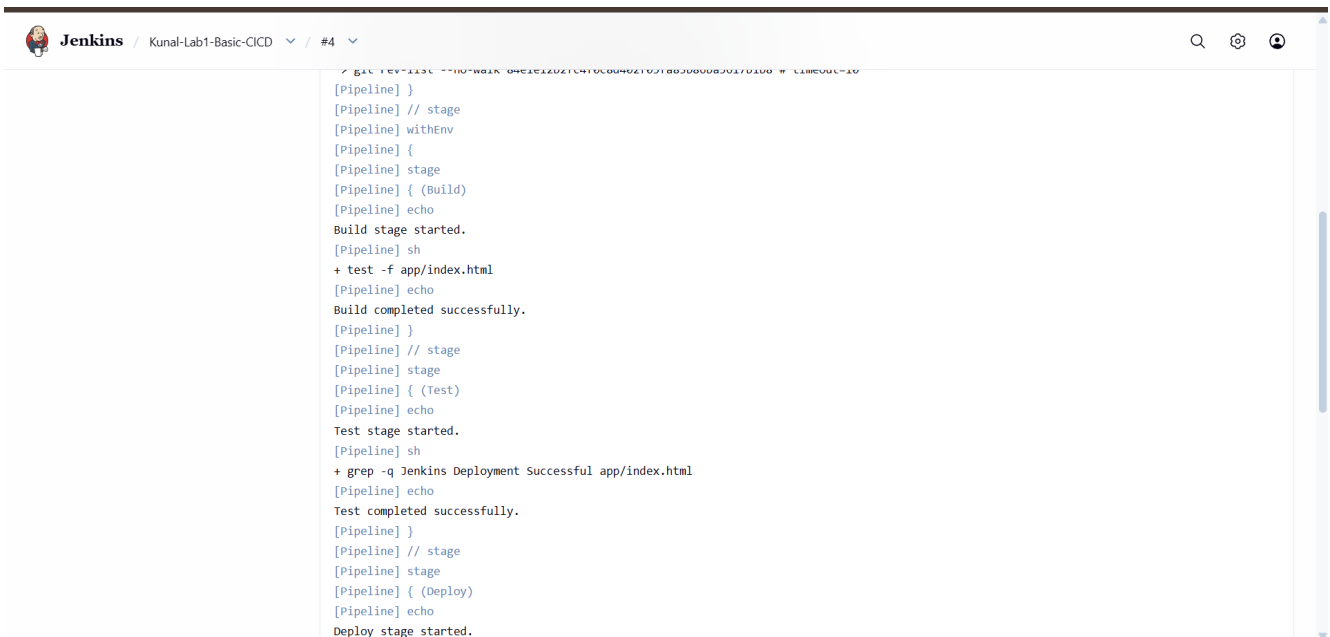
Download

Copy

View as plain text

```
Started by user Kunal Prajapati
Obtained Jenkinsfile from git https://github.com/kunalprajapati14042005/lab1-devops-version-control.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Kunal-Lab1-Basic-CICD
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/Kunal-Lab1-Basic-CICD/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/kunalprajapati14042005/lab1-devops-version-control.git # timeout=10
Fetching upstream changes from https://github.com/kunalprajapati14042005/lab1-devops-version-control.git
> git --version # timeout=10
> git --version # 'git version 2.43.0'
> git fetch --tags --force --progress -- https://github.com/kunalprajapati14042005/lab1-devops-version-control.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 84e1e12b2fc4f0c8d402f05fa83b86ba5617b1b8 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 84e1e12b2fc4f0c8d402f05fa83b86ba5617b1b8 # timeout=10
Commit message: "Add Jenkins CI/CD pipeline"
```

Figure 3. Jenkins console output showing GitHub checkout and pipeline initialization.

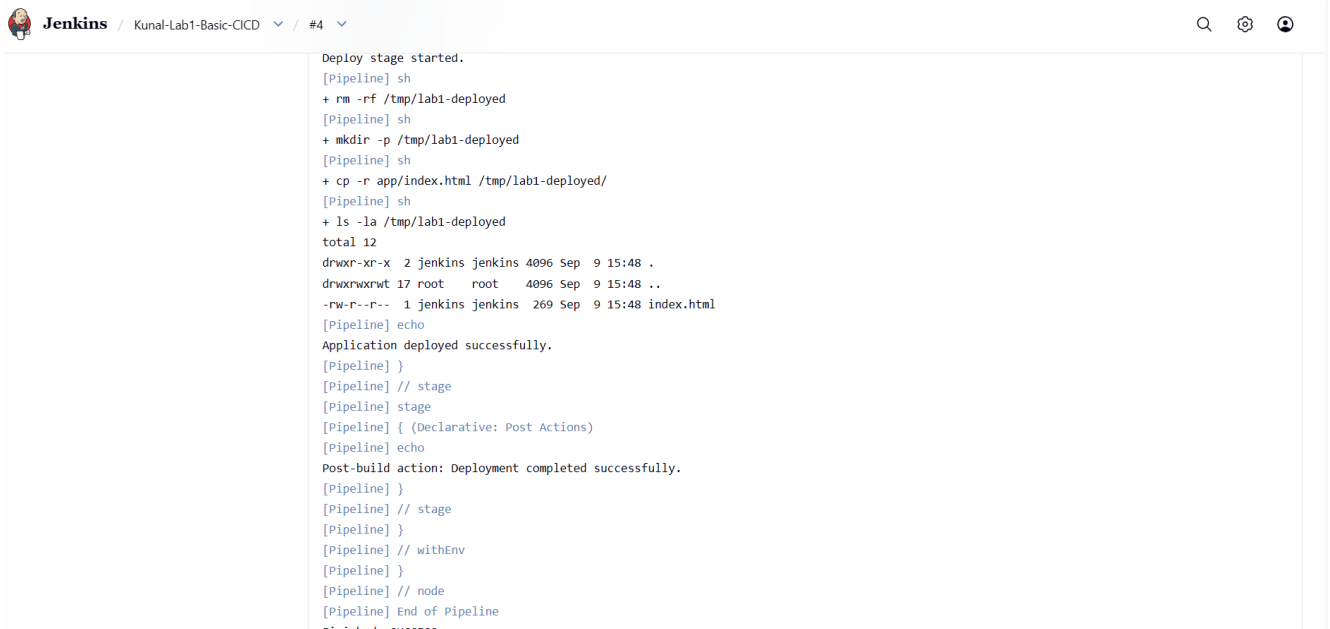


```
Jenkins / Kunal-Lab1-Basic-CICD / #4
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Build stage started.
[Pipeline] sh
+ test -f app/index.html
[Pipeline] echo
Build completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Test stage started.
[Pipeline] sh
+ grep -q Jenkins Deployment Successful app/index.html
[Pipeline] echo
Test completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploy stage started.
```

Figure 4. Build and Test stages completing successfully for Build #4.

4. Automated Deployment and Post-build Action

The Deploy stage removes the previous deployment directory, creates /tmp/lab1-deployed, and copies app/index.html into it. The Jenkins console confirms the deployed file and the post-build success action.



```
Jenkins / Kunal-Lab1-Basic-CICD / #4
Deploy stage started.
[Pipeline] sh
+ rm -rf /tmp/lab1-deployed
[Pipeline] sh
+ mkdir -p /tmp/lab1-deployed
[Pipeline] sh
+ cp -r app/index.html /tmp/lab1-deployed/
[Pipeline] sh
+ ls -la /tmp/lab1-deployed
total 12
drwxr-xr-x  2 jenkins jenkins 4096 Sep  9 15:48 .
drwxrwxrwt 17 root    root    4096 Sep  9 15:48 ..
-rw-r--r--  1 jenkins jenkins 269 Sep  9 15:48 index.html
[Pipeline] echo
Application deployed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Post-build action: Deployment completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 5. Jenkins console showing the deployment steps, deployed index.html, and successful post-build action.

5. Deployed Application Verification

The deployed application was served from the deployment directory and opened successfully in the browser at <http://localhost:8000>. The page displays the deployment confirmation message, providing direct evidence that the deployed artifact is accessible after the Jenkins deployment step.

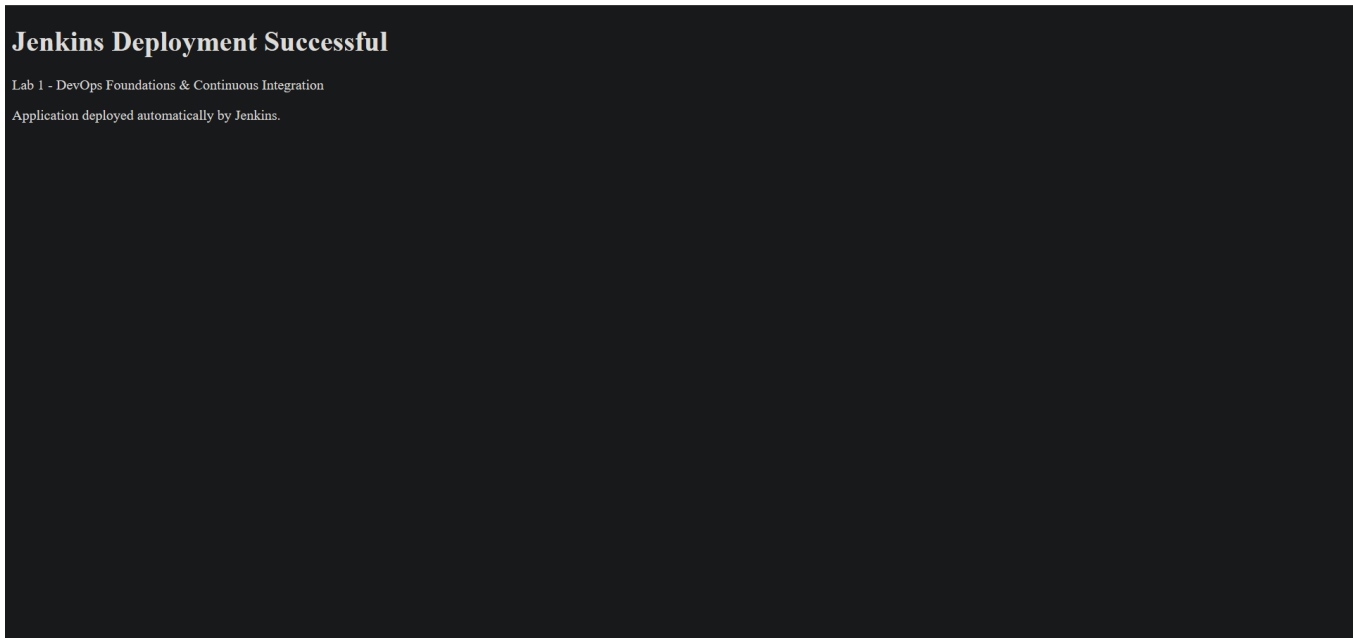


Figure 6. Browser verification of the Jenkins-deployed web application.

6. Automation Results and Observations

The workflow demonstrates an end-to-end CI/CD process in Jenkins. Source code is maintained in GitHub, Jenkins is configured with an automated polling trigger, the pipeline validates the application during Build and Test, and the Deploy stage places the application into a dedicated deployment directory.

Observations

1. GitHub provides centralized source control for the pipeline.
2. Jenkins automatically retrieves the repository and Jenkinsfile before executing the pipeline.
3. Build and Test stages provide basic validation before deployment.
4. The Deploy stage creates a fresh deployment directory and copies the application artifact into it.
5. The post-build action confirms successful completion, and the browser verification demonstrates that the deployed artifact is accessible.

7. Conclusion

A working CI/CD workflow was implemented using GitHub and Jenkins. The repository was checked out successfully, the Build and Test stages completed, the application was deployed automatically, and the resulting web page was successfully verified in a browser. The workflow therefore documents the architecture, stages, automation result, and observations required for this CI/CD report.

Evidence note: The report uses the current Jenkins trigger configuration, Build #4 console evidence, deployment output, and browser verification collected during the lab work.